УДК 004.4'42

Автоматическая верификация оптимизаций LuaJIT с использованием SMT-решателя

***А. С. Чуркин*[1], *С. Бронников*[2], *С. Каплун*[2]**

[1]Московский физико-технический институт (национальный исследовательский университет)
[2]ООО «VK»

Компиляторы — категория программного обеспечения, где важна корректность работы. Ошибки в компиляторах приводят к трудно диагностируемым сбоям в пользовательском коде и могут потребовать значительного времени для локализации. В современных компиляторах применяются оптимизаторы со сложными алгоритмами, которые требуют надёжного тестирования.

Исследования в области повышения надёжности компиляторов ведутся начиная с конца 2000-х годов. В 2009 году был создан первый полностью формально верифицированный компилятор C (CompCert), в 2013 был представлен фаззер AFL, широко применяемый для тестирования компиляторов. В 2021 году вышла статья по Alive2 [1], посвящённая верификации оптимизаций LLVM IR. Аналогичные подходы реализованы для GCC [2] и Python [3].

В данной работе демонстрируется реализация автоматического верификатора для проверки корректности JIT-оптимизаций в компиляторе LuaJIT. Работа опирается на два внешних инструмента: SMT-решатель и фаззер LuaJIT. SMT-решатель Z3 [4] используется для проверки выполнимости формул в теориях Floating Points, Arrays, BitVectors. Фаззер LuaJIT, разработанный в компании «VK», генерирует корректные входные программы на Lua.

Предложенный подход основан на проверке эквивалентности (translation validation): для программы до и после применения оптимизаций строятся SMT-формулы, описывающие её семантику, и проверяется, что наблюдаемое поведение не изменилось. В отличие от дифференциального тестирования верификация через SMT-решатель позволяет проверить корректность на всех допустимых входах, а не на конкретных тестовых данных. Программа представляется как набор ограничений над символьными переменными (см. Листинг 1).

```
local function foo(a, b)
   local x = a + b
   local y = a - b
   return x, y
end
```

```
assert(a == vm_stack(0))
assert(b == vm_stack(1))
assert(x == a + b)
assert(y == a - b)
assert(result == (x, y))
```

Листинг 1. Пример символического исполнения.

В рамках данной работы был реализован верификатор, кодирующий инструкции SSA IR LuaJIT [5] на языке SMT-LIB с сохранением их семантики. А именно, были реализованы алгоритмы для кодирования SSA IR LuaJIT-а на языке SMTLIB с сохранением их семантики и особенностей. Для корректного сопоставления формул была использована специфика трасс LuaJIT, снимков состояния (snapshots) и причин выхода с трасс в интерпретацию. По входной программе верификатор строит SMT-формулу. Сделав это для разных уровней оптимизаций и правильно объединив формулы, можно закодировать в них вопрос: «Существует ли вход, на котором эти формулы различны?» SMT-решатель либо вернёт входные данные, на которых эти формулы различны, либо то, что такого входа не существует (оптимизация ничего не нарушила).

Результатом работы является полностью автоматический инструмент для проверки трасс исходного и оптимизированного кода на языке LuaJIT.

Литература

1. Nuno P. Lopes. [et al.]. Alive2: Bounded Translation Validation for LLVM. // 2021.
2. Krister Walfridsson. GCC Translation Validation. // 2022.
3. Bolz-Tereick. Finding JIT Optimizer Bugs using SMT Solvers and Fuzzing. // 2022.
4. de Moura, L.& Bjørner, N. (2008). Z3: An Efficient SMT Solver. // 2008. TACAS (pp. 337-340)
5. Dario d’Andrea, Laurent Deniau. Tecnical Documentation TRACE-BASED JUST-IN-TIME COMPILER LuaJIT. // 2020